# causal forest

2026-09-30

## Contents


**1 Setup and data preparation** **1**

**2 Diagnostic tests** **3**

2.1 Missing values across waves . . . . . 3

2.2 Sample and treatment coding . . . . . 4

2.3 Name penalty by wave . . . . . 5

**3 Causal forests** **5**

3.1 Specifications . . . . . 5

3.2 Fit . . . . . 6

3.3 Overlap / propensity check . . . . . 6

**4 Causal forest results** **7**

4.1 Average treatment effect . . . . . 7

4.2 Variable importance . . . . . 8

4.3 Estimated effects (CATE) . . . . . 9

**5 Validation of causal forest results** **12**

5.1 test_calibration . . . . . 12

5.2 RATE / TOC with a firm-level split . . . . . 14

5.3 Combined reading (rule fixed in advance) . . . . . 17

5.4 Tuning robustness . . . . . 18


This document runs in four stages: **diagnostic tests** on the sample, **causal forests** (fit and overlap check), **causal forest results** (ATE, variable importance, estimated effects) and **validation** of those results (calibration, RATE/TOC, combined verdict, tuning robustness). Covariate balance does not depend on any forest, so it lives in a separate file: `Causal Forest Balance Table.Rmd`. Both files read the analysis sample saved by `cleaned_data.Rmd` (`cleaned data.Rda`), so knit that one first.

# 1 Setup and data preparation

```r
required_packages <- c("estimatr", "dplyr", "tidyr", "grf", "knitr")
missing_packages  <- setdiff(required_packages, rownames(installed.packages()))
if (length(missing_packages) > 0) {
  install.packages(missing_packages, repos = "https://cloud.r-project.org")
}
library(estimatr)
library(dplyr)
library(tidyr)
library(grf)
library(knitr)
```

```r
# cleaned_data.Rmd builds and saves the analysis sample (df_int1)
rda_path <- "cleaned data.Rda"
if (!file.exists(rda_path)) {
  stop("Cleaned data not found. Knit cleaned_data.Rmd first to create: ",
       normalizePath(rda_path, mustWork = FALSE))
}
loaded <- load(rda_path)
stopifnot("df_int1" %in% loaded)
```

```r
# Pre-treatment resume characteristics. Keep this order: grf picks split variables by
# column position, so the forests only reproduce exactly with the same order.
all_pretreat_cov <- c(
  "language_skills",
  "female",
  "ba_quality",
  "extracurricular_skills",
  "ma",
  "same_exp",
  "exp_highquality"
)

# df_int1 comes from cleaned_data.Rmd: Canadian-named (control) and Chinese/Indian/
# Pakistani-named (treated) resumes, D = 1 for a foreign name, and reference,
# accreditation and legal (wave 2 only) dropped.
# Missing values are NOT recoded there: same_exp is NA for every resume in wave 2 (see the
# missing-value check below). grf uses missing covariate values directly; the tables that
# need observed values (balance, subgroups, BLP) use waves 0-1 for same_exp.

# The wave is saved as a factor; use a numeric code (0, 1, 2) so it can enter X.
df_int1 <- df_int1 %>%
  mutate(
    wave_1 = as.integer(fall_data == 1),
    wave_2 = as.integer(fall_data == 2)
  )

Y_all   <- as.numeric(df_int1$callback)   # outcome: callback
W_all   <- as.numeric(df_int1$D)          # treatment: foreign-sounding name
firm_id <- df_int1$firmid                 # cluster: employer
waves   <- sort(unique(df_int1$fall_data))
```

# 2 Diagnostic tests

These checks use the data only (no forest). For covariate balance and wave composition see `Causal Forest Balance Table.Rmd`.

## 2.1 Missing values across waves

The cleaned data keeps missing values as they are, so this shows what is really missing: only same_exp has gaps (all of wave 2).

```
vars <- c("type", all_pretreat_cov)

counts_all <- df_int1 %>%
  mutate(across(all_of(vars), as.character)) %>%
  pivot_longer(all_of(vars), names_to = "variable", values_to = "value") %>%
  mutate(variable = factor(variable, levels = vars)) %>%
  group_by(fall_data, variable) %>%
  summarise(
    with_value = sum(!is.na(value) & trimws(value) != ""),
    empty      = sum(is.na(value) | trimws(value) == ""),
    .groups = "drop"
  ) %>%
  left_join(df_int1 %>% count(fall_data, name = "total_rows"), by = "fall_data") %>%
  relocate(total_rows, .after = fall_data) %>%
  arrange(fall_data, variable)

print(counts_all, n = Inf)
```

```
## # A tibble: 24 x 5
##    fall_data total_rows variable               with_value empty
##    <fct>          <int> <fct>                       <int> <int>
##  1 0               1902 type                         1902     0
##  2 0               1902 language_skills              1902     0
##  3 0               1902 female                       1902     0
##  4 0               1902 ba_quality                   1902     0
##  5 0               1902 extracurricular_skills       1902     0
##  6 0               1902 ma                           1902     0
##  7 0               1902 same_exp                     1902     0
##  8 0               1902 exp_highquality              1902     0
##  9 1               1495 type                         1495     0
## 10 1               1495 language_skills              1495     0
## 11 1               1495 female                       1495     0
## 12 1               1495 ba_quality                   1495     0
## 13 1               1495 extracurricular_skills       1495     0
## 14 1               1495 ma                           1495     0
## 15 1               1495 same_exp                     1495     0
## 16 1               1495 exp_highquality              1495     0
## 17 2               2260 type                         2260     0
## 18 2               2260 language_skills              2260     0
## 19 2               2260 female                       2260     0
## 20 2               2260 ba_quality                   2260     0
## 21 2               2260 extracurricular_skills       2260     0
```

```
## 22 2              2260 ma                          2260     0
## 23 2              2260 same_exp                       0  2260
## 24 2              2260 exp_highquality             2260     0
```

## 2.2 Sample and treatment coding

```
cat("N:", nrow(df_int1), "(expected 5,657)\n")
```

```
## N: 5657 (expected 5,657)
```

```
cat("Firms:", length(unique(df_int1$firmid)), "\n")
```

```
## Firms: 3269
```

```
cat("Resumes per firm:\n")
```

```
## Resumes per firm:
```

```
print(summary(as.numeric(table(df_int1$firmid))))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    1.00    2.00    1.73    2.00    2.00
```

```
cat("\nTreatment counts (expected 3,026 control / 2,631 treated):\n")
```

```
##
## Treatment counts (expected 3,026 control / 2,631 treated):
```

```
print(table(D = df_int1$D))
```

```
## D
##    0    1
## 3026 2631
```

```
cat("\nName ethnicity by treatment (D must match the coding exactly):\n")
```

```
##
## Name ethnicity by treatment (D must match the coding exactly):
```

```
print(table(ethnicity = as.character(df_int1$name_ethnicity), D = df_int1$D))
```

```
##            D
## ethnicity     0    1
##   Canada   3026    0
##   Chinese     0  998
##   Indian      0 1167
##   Pakistani   0  466
```

```r
cat("\nWave by treatment:\n")
```

```
##
## Wave by treatment:
```

```r
print(table(wave = df_int1$fall_data, D = df_int1$D))
```

```
##     D
## wave    0    1
##    0  953  949
##    1 1029  466
##    2 1044 1216
```

### 2.3 Name penalty by wave

Does the name penalty itself differ by wave? Plain regression of callback on D within each wave, so no forest is involved.

```r
ate_wave <- bind_rows(lapply(waves, function(k) {
  dat   <- filter(df_int1, fall_data == k)
  dat$y <- as.numeric(dat$callback)
  fit   <- lm_robust(y ~ D, data = dat, clusters = firmid, se_type = "CR2")
  data.frame(wave      = k,
             n         = nrow(dat),
             effect_pp = 100 * unname(coef(fit)["D"]),
             se_pp     = 100 * unname(fit$std.error["D"]),
             lo_pp     = 100 * unname(fit$conf.low["D"]),
             hi_pp     = 100 * unname(fit$conf.high["D"]))
}))

kable(ate_wave, digits = 2,
      caption = "Name penalty by wave (percentage points, firm-clustered CR2)")
```

Table 1: Name penalty by wave (percentage points, firm-clustered CR2)

| wave | n | effect_pp | se_pp | lo_pp | hi_pp |
|---|---|---|---|---|---|
| 0 | 1902 | -4.36 | 1.17 | -6.66 | -2.06 |
| 1 | 1495 | -4.79 | 1.36 | -7.47 | -2.11 |
| 2 | 2260 | -4.29 | 1.11 | -6.46 | -2.13 |

## 3 Causal forests

### 3.1 Specifications

All three forests use the same outcome (callback), treatment (D, foreign-sounding name) and clustering (firmid). They differ only in the covariates X:

- **cf1**: three moderators (language skills, extracurricular skills, multinational experience).
- **cf1w**: the three moderators plus the wave indicator, so it is the like-for-like comparison with the wave-adjusted regressions.
- **cf2**: all pre-treatment covariates plus the wave indicator.

All three are validated and reported; none is picked after the fact. same_exp is NA for all of wave 2 and is passed to grf as is (grf handles missing covariate values natively).

```
# TO ASK: MUST WE INCLUDE FALL_DATA?
moderators <- c("language_skills", "extracurricular_skills", "same_exp")

covariate_sets <- list(
  "cf1 (3 moderators)"         = moderators,
  "cf1w (3 moderators + wave)" = c(moderators, "wave_1","wave_2"),
  "cf2 (full covariates)"      = c(all_pretreat_cov, "wave_1","wave_2")
)

# Short labels (cf1, cf1w, cf2) for the wide validation tables further down
short_name <- function(x) sub(" .*", "", x)
```

## 3.2 Fit

```
fit_forest <- function(vars) {
  X <- df_int1 %>% select(all_of(vars)) %>% as.matrix()
  # grf draws its forest seed from R's random number generator, so reset the seed
  # immediately before every fit to make each forest reproducible.
  set.seed(4101)
  list(cf = causal_forest(X, Y_all, W_all, clusters = firm_id), X = X)
}

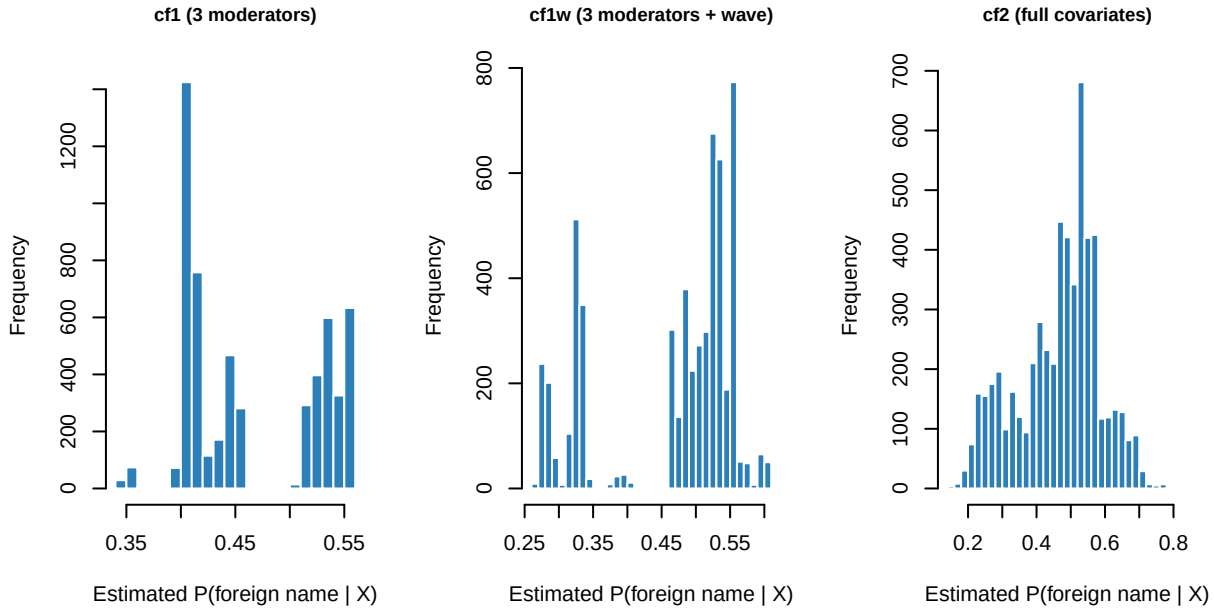
forests <- lapply(covariate_sets, fit_forest)

cf1   <- forests[["cf1 (3 moderators)"]]$cf
cf2   <- forests[["cf2 (full covariates)"]]$cf
X_cf2 <- forests[["cf2 (full covariates)"]]$X
```

## 3.3 Overlap / propensity check

Because name ethnicity is experimentally randomized within a job posting, the estimated propensity scores (`W.hat`) should cluster fairly tightly around the actual treatment share rather than displaying the extreme values that signal confounding.

```
par(mfrow = c(1, 3))
for (nm in names(forests)) {
  hist(forests[[nm]]$cf$W.hat, breaks = 30, col = "#2c7fb8", border = "white",
       main = nm, cex.main = 0.9, xlab = "Estimated P(foreign name | X)")
}
```




```
par(mfrow = c(1, 1))
```

# 4 Causal forest results

## 4.1 Average treatment effect

```
ate_tab <- bind_rows(lapply(names(forests), function(nm) {
  a <- average_treatment_effect(forests[[nm]]$cf, target.sample = "all")
  data.frame(forest    = nm,
             effect_pp = 100 * a[["estimate"]],
             se_pp     = 100 * a[["std.err"]])
})) %>%
  mutate(lo_pp = effect_pp - 1.96 * se_pp, hi_pp = effect_pp + 1.96 * se_pp)

kable(ate_tab, digits = 3,
      caption = "Average treatment effect by forest (percentage points)")
```

Table 2: Average treatment effect by forest (percentage points)

| forest | effect_pp | se_pp | lo_pp | hi_pp |
|---|---|---|---|---|
| cf1 (3 moderators) | -3.996 | 0.691 | -5.350 | -2.641 |
| cf1w (3 moderators + wave) | -4.377 | 0.698 | -5.745 | -3.009 |
| cf2 (full covariates) | -4.462 | 0.742 | -5.917 | -3.006 |

```
for (i in seq_len(nrow(ate_tab))) {
  cat(sprintf(
```

```
    paste0("**Estimated ATE, %s:** having a foreign/ethnic-sounding name changes the ",
           "callback probability by **%.3f** percentage points (SE = %.3f), ",
           "on average across the analysis sample.\n\n"),
    ate_tab$forest[i], ate_tab$effect_pp[i], ate_tab$se_pp[i]
  ))
}
```

**Estimated ATE, cf1 (3 moderators):** having a foreign/ethnic-sounding name changes the callback probability by **-3.996** percentage points (SE = 0.691), on average across the analysis sample.

**Estimated ATE, cf1w (3 moderators + wave):** having a foreign/ethnic-sounding name changes the callback probability by **-4.377** percentage points (SE = 0.698), on average across the analysis sample.

**Estimated ATE, cf2 (full covariates):** having a foreign/ethnic-sounding name changes the callback probability by **-4.462** percentage points (SE = 0.742), on average across the analysis sample.

## 4.2 Variable importance

Which covariates does each forest use most to split on (i.e., which ones drive the most predictable heterogeneity)?

```
vi_tab <- bind_rows(lapply(names(forests), function(nm) {
  data.frame(forest     = nm,
             variable   = colnames(forests[[nm]]$X),
             importance = as.numeric(variable_importance(forests[[nm]]$cf)))
})) %>%
  arrange(factor(forest, levels = names(forests)), desc(importance))

kable(vi_tab, digits = 3, caption = "Causal forest variable importance")
```

Table 3: Causal forest variable importance

| forest | variable | importance |
|---|---|---|
| cf1 (3 moderators) | same_exp | 0.412 |
| cf1 (3 moderators) | extracurricular_skills | 0.325 |
| cf1 (3 moderators) | language_skills | 0.263 |
| cf1w (3 moderators + wave) | extracurricular_skills | 0.311 |
| cf1w (3 moderators + wave) | same_exp | 0.290 |
| cf1w (3 moderators + wave) | language_skills | 0.213 |
| cf1w (3 moderators + wave) | wave_1 | 0.122 |
| cf1w (3 moderators + wave) | wave_2 | 0.064 |
| cf2 (full covariates) | female | 0.197 |
| cf2 (full covariates) | ba_quality | 0.190 |
| cf2 (full covariates) | same_exp | 0.147 |
| cf2 (full covariates) | extracurricular_skills | 0.133 |
| cf2 (full covariates) | language_skills | 0.098 |
| cf2 (full covariates) | exp_highquality | 0.094 |
| cf2 (full covariates) | ma | 0.078 |
| cf2 (full covariates) | wave_1 | 0.038 |
| cf2 (full covariates) | wave_2 | 0.024 |

## 4.3 Estimated effects (CATE)

The forest's own output: a predicted treatment effect for every resume. Read these as descriptive/exploratory unless the validation section supports heterogeneity.

### 4.3.1 Distribution of predicted effects

```
# Out-of-bag predictions: each resume is predicted by trees that never saw it
cate_df <- bind_rows(lapply(names(forests), function(nm) {
  p <- predict(forests[[nm]]$cf, estimate.variance = TRUE)
  data.frame(forest  = nm,
             tau_hat = p$predictions,
             tau_se  = sqrt(p$variance.estimates))
}))

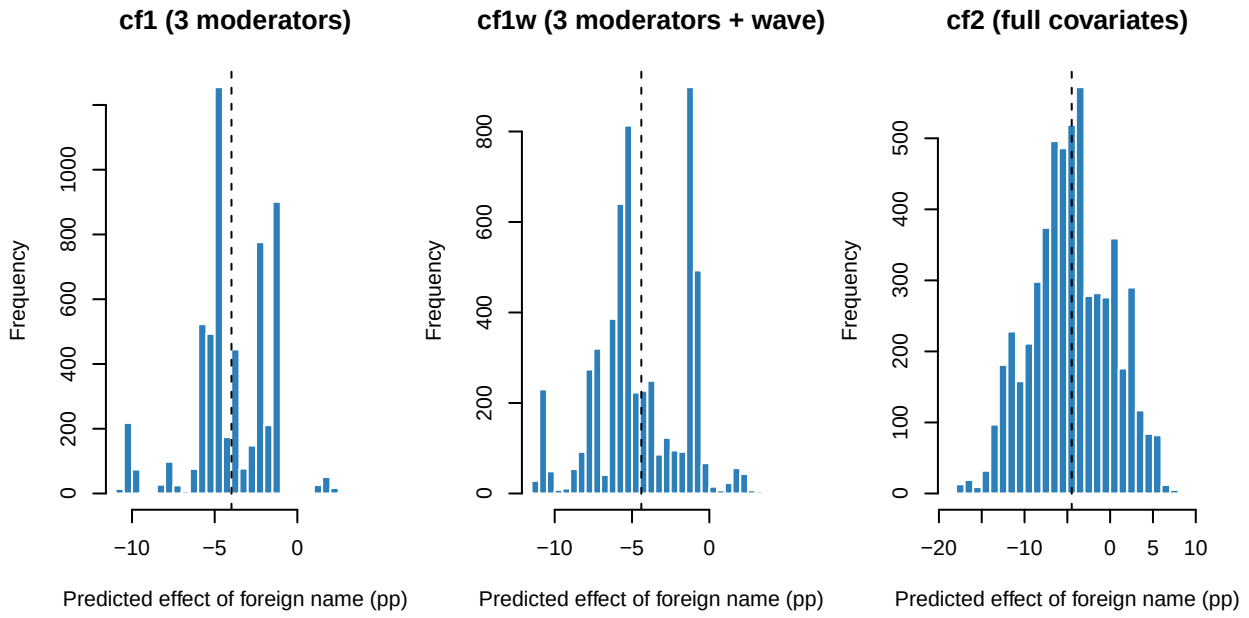
cate_summary <- cate_df %>%
  group_by(forest) %>%
  summarise(mean_pp    = 100 * mean(tau_hat),
            sd_pp      = 100 * sd(tau_hat),
            min_pp     = 100 * min(tau_hat),
            p25_pp     = 100 * quantile(tau_hat, .25),
            median_pp  = 100 * median(tau_hat),
            p75_pp     = 100 * quantile(tau_hat, .75),
            max_pp     = 100 * max(tau_hat),
            mean_se_pp = 100 * mean(tau_se),
            .groups = "drop")

kable(cate_summary, digits = 2,
      caption = "Distribution of predicted treatment effects (pp, out-of-bag)")
```

Table 4: Distribution of predicted treatment effects (pp, out-of-bag)

| forest | mean_pp | sd_pp | min_pp | p25_pp | median_pp | p75_pp | max_pp | mean_se_pp |
|---|---|---|---|---|---|---|---|---|
| cf1 (3 moderators) | -3.98 | 2.34 | -10.79 | -5.09 | -4.49 | -2.09 | 3.13 | 2.33 |
| cf1w (3 moderators + wave) | -4.40 | 2.94 | -11.48 | -6.06 | -5.08 | -1.18 | 3.69 | 3.08 |
| cf2 (full covariates) | -4.49 | 4.67 | -18.77 | -7.47 | -4.47 | -1.04 | 8.51 | 4.62 |

```
par(mfrow = c(1, 3))
for (nm in names(forests)) {
  tau_pp <- 100 * cate_df$tau_hat[cate_df$forest == nm]
  hist(tau_pp, breaks = 30, col = "#2c7fb8", border = "white", main = nm,
       xlab = "Predicted effect of foreign name (pp)")
  abline(v = mean(tau_pp), lty = 2)
}
```




```
par(mfrow = c(1, 1))
```

### 4.3.2 Effect by subgroup (cf2)

Doubly robust ATE within each subgroup. same_exp is only observed in waves 0-1, so its two rows cover those waves only.

```
subset_ate <- function(cf, idx, label) {
  idx <- which(idx)    # drops resumes where the covariate is missing (NA)
  a <- average_treatment_effect(cf, subset = idx)
  data.frame(group     = label,
             n         = length(idx),
             effect_pp = 100 * a[["estimate"]],
             se_pp     = 100 * a[["std.err"]])
}

subgroups <- c(
  lapply(all_pretreat_cov, function(v) {
    bind_rows(subset_ate(cf2, df_int1[[v]] == 1, paste(v, "= 1")),
              subset_ate(cf2, df_int1[[v]] == 0, paste(v, "= 0")))
  }),
  lapply(waves, function(k) {
    subset_ate(cf2, df_int1$fall_data == k, paste("wave =", k))
  })
)

sub_tab <- bind_rows(subgroups) %>%
  mutate(lo_pp = effect_pp - 1.96 * se_pp, hi_pp = effect_pp + 1.96 * se_pp)

kable(sub_tab, digits = 2, caption = "Forest-based effect by subgroup (cf2, pp)")
```

Table 5: Forest-based effect by subgroup (cf2, pp)

| group | n | effect_pp | se_pp | lo_pp | hi_pp |
|---|---|---|---|---|---|
| language_skills = 1 | 1424 | -5.42 | 1.84 | -9.02 | -1.82 |
| language_skills = 0 | 4233 | -4.14 | 0.89 | -5.88 | -2.40 |
| female = 1 | 2876 | -6.29 | 1.21 | -8.66 | -3.93 |
| female = 0 | 2781 | -2.57 | 1.14 | -4.80 | -0.33 |
| ba_quality = 1 | 3457 | -2.91 | 1.02 | -4.91 | -0.92 |
| ba_quality = 0 | 2200 | -6.89 | 1.35 | -9.54 | -4.24 |
| extracurricular_skills = 1 | 3417 | -3.43 | 1.07 | -5.53 | -1.33 |
| extracurricular_skills = 0 | 2240 | -6.03 | 1.39 | -8.75 | -3.32 |
| ma = 1 | 1142 | -4.39 | 2.00 | -8.30 | -0.48 |
| ma = 0 | 4515 | -4.48 | 0.86 | -6.17 | -2.79 |
| same_exp = 1 | 1498 | -2.96 | 1.69 | -6.28 | 0.35 |
| same_exp = 0 | 1899 | -5.68 | 1.36 | -8.35 | -3.01 |
| exp_highquality = 1 | 1837 | -3.44 | 1.59 | -6.56 | -0.32 |
| exp_highquality = 0 | 3820 | -4.95 | 0.98 | -6.88 | -3.02 |
| wave = 0 | 1902 | -4.12 | 1.27 | -6.62 | -1.62 |
| wave = 1 | 1495 | -4.94 | 1.48 | -7.83 | -2.05 |
| wave = 2 | 2260 | -4.43 | 1.16 | -6.71 | -2.15 |

#### 4.3.3 Best linear projection (cf2)

Cluster-robust test of which covariates linearly shift the effect. Uses resumes with every covariate observed, i.e. waves 0-1 (same_exp is missing in wave 2).

```
blp <- best_linear_projection(cf2, X_cf2, subset = which(complete.cases(X_cf2)))
print(blp)
```

```
##
## Best linear projection of the conditional average treatment effect.
## Confidence intervals are cluster- and heteroskedasticity-robust (HC3):
##
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            -0.0777012  0.0290601 -2.6738 0.007535 **
## language_skills         0.0263285  0.0307858  0.8552 0.392492
## female                 -0.0250888  0.0243798 -1.0291 0.303514
## ba_quality              0.0484939  0.0235739  2.0571 0.039753 *
## extracurricular_skills  0.0125717  0.0253678  0.4956 0.620226
## ma                     -0.0026636  0.0300489 -0.0886 0.929370
## same_exp                0.0250738  0.0313008  0.8011 0.423152
## exp_highquality         0.0011321  0.0344206  0.0329 0.973764
## wave_1                 -0.0056990  0.0194529 -0.2930 0.769567
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

#### 4.3.4 Predicted effect by covariate cell (cf1)

cf1 has only a few distinct covariate combinations, so show them directly (same_exp = NA is wave 2).

```r
cf1_cells <- df_int1 %>%
  mutate(tau_hat = predict(cf1)$predictions) %>%
  group_by(language_skills, extracurricular_skills, same_exp) %>%
  summarise(n = n(), effect_pp = 100 * mean(tau_hat), .groups = "drop") %>%
  arrange(effect_pp)

kable(cf1_cells, digits = 2, caption = "cf1: predicted effect by covariate cell (pp)")
```

Table 6: cf1: predicted effect by covariate cell (pp)

| language_skills | extracurricular_skills | same_exp | n | effect_pp |
|---|---|---|---|---|
| 1 | 0 | NA | 308 | -10.14 |
| 1 | 0 | 0 | 162 | -7.74 |
| 0 | 0 | 0 | 597 | -5.85 |
| 0 | 0 | NA | 571 | -5.12 |
| 1 | 1 | NA | 422 | -4.68 |
| 0 | 1 | 0 | 900 | -4.66 |
| 0 | 0 | 1 | 494 | -3.78 |
| 1 | 1 | 1 | 184 | -2.90 |
| 1 | 1 | 0 | 240 | -2.26 |
| 0 | 1 | 1 | 712 | -2.12 |
| 0 | 1 | NA | 959 | -1.36 |
| 1 | 0 | 1 | 108 | 1.65 |

# 5 Validation of causal forest results

## 5.1 test_calibration

The differential forest prediction is the number that matters. grf's p-value on it is one-sided (is the coefficient above zero?). Near 1 means well calibrated, between 0 and 1 means the forest overstates the spread, and at or below 0 means the spread isn't tracking real differences.

```r
calibration <- lapply(forests, function(f) test_calibration(f$cf))

for (nm in names(calibration)) {
  cat("\n====", nm, "====\n")
  print(calibration[[nm]])
}
```

```
##
## ==== cf1 (3 moderators) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                 Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction          1.007059   0.173210  5.8141 3.215e-09 ***
## differential.forest.prediction -0.052759   0.373814 -0.1411    0.5561
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## ==== cf1w (3 moderators + wave) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                               Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction         0.99453    0.15804  6.2931 1.673e-10 ***
## differential.forest.prediction -0.18527    0.30052 -0.6165    0.7312
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## ==== cf2 (full covariates) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                               Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction         0.994295   0.157222  6.3241 1.371e-10 ***
## differential.forest.prediction 0.021829   0.189598  0.1151    0.4542
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cal_tab <- bind_rows(lapply(names(calibration), function(nm) {
  tc <- calibration[[nm]]
  data.frame(
    forest           = nm,
    mean_est         = tc["mean.forest.prediction", "Estimate"],
    mean_se          = tc["mean.forest.prediction", "Std. Error"],
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))

kable(cal_tab, digits = 3, caption = "test_calibration results")
```

Table 7: test_calibration results

| forest | mean_est | mean_se | diff_est | diff_se | diff_p_one_sided |
|---|---|---|---|---|---|
| cf1 (3 moderators) | 1.007 | 0.173 | -0.053 | 0.374 | 0.556 |
| cf1w (3 moderators + wave) | 0.995 | 0.158 | -0.185 | 0.301 | 0.731 |
| cf2 (full covariates) | 0.994 | 0.157 | 0.022 | 0.190 | 0.454 |

## 5.2 RATE / TOC with a firm-level split

Half of the firms train a forest that produces the priority scores; the other half trains a separate evaluation forest. The split is by firm, not by resume, so resumes from one employer never sit on both sides. AUTOC is the primary measure and QINI the secondary one, chosen in advance. The priority score is the forest's predicted effect with no sign flip, so the top of the ranking is the group with the smallest predicted penalty (the least negative effects), and a positive RATE means the ranking sorts correctly.

Seed 1 is the headline split, fixed in advance. Seeds 2 to `n_seeds` are stability checks only. This chunk fits many forests and can take a few minutes.

```
n_seeds <- 5

make_split <- function(firm, seed) {
  set.seed(seed)    # set immediately before sampling so the split is reproducible
  f <- unique(firm)
  firm %in% sample(f, floor(length(f) / 2))
}

# Train on one half of the firms and evaluate on the other half.
# `targets` picks the RATE measures to compute (one list element each: autoc, qini).
# `...` passes extra arguments (e.g. tuning settings) to causal_forest.
# The RATE standard errors are bootstrapped, so computing extra targets uses up
# random draws; the tuning check asks for AUTOC only so its draws stay unchanged.
run_rate <- function(X, Y, W, firm, train, seed, targets = c("AUTOC", "QINI"), ...) {
  cf_train <- causal_forest(X[train, , drop = FALSE],  Y[train],  W[train],
                            clusters = firm[train],  seed = seed, ...)
  cf_eval  <- causal_forest(X[!train, , drop = FALSE], Y[!train], W[!train],
                            clusters = firm[!train], seed = seed, ...)
  priorities <- predict(cf_train, X[!train, , drop = FALSE])$predictions
  rates <- lapply(targets, function(tg) {
    rank_average_treatment_effect(cf_eval, priorities, target = tg)
  })
  setNames(rates, tolower(targets))
}

rate_res <- lapply(forests, function(f) {
  out <- lapply(seq_len(n_seeds), function(s) {
    train <- make_split(firm_id, s)
    run_rate(f$X, Y_all, W_all, firm_id, train, seed = s)
  })
  names(out) <- seq_len(n_seeds)
  out
})

rate_tab <- bind_rows(lapply(names(rate_res), function(nm) {
  bind_rows(lapply(names(rate_res[[nm]]), function(s) {
    a <- rate_res[[nm]][[s]]$autoc
    q <- rate_res[[nm]][[s]]$qini
    data.frame(forest = nm, seed = as.integer(s), headline = (s == "1"),
               autoc = a$estimate, autoc_se = a$std.err,
               autoc_lo = a$estimate - 1.96 * a$std.err,
               autoc_hi = a$estimate + 1.96 * a$std.err,
               qini = q$estimate, qini_se = q$std.err)
  }))
```

```
}))

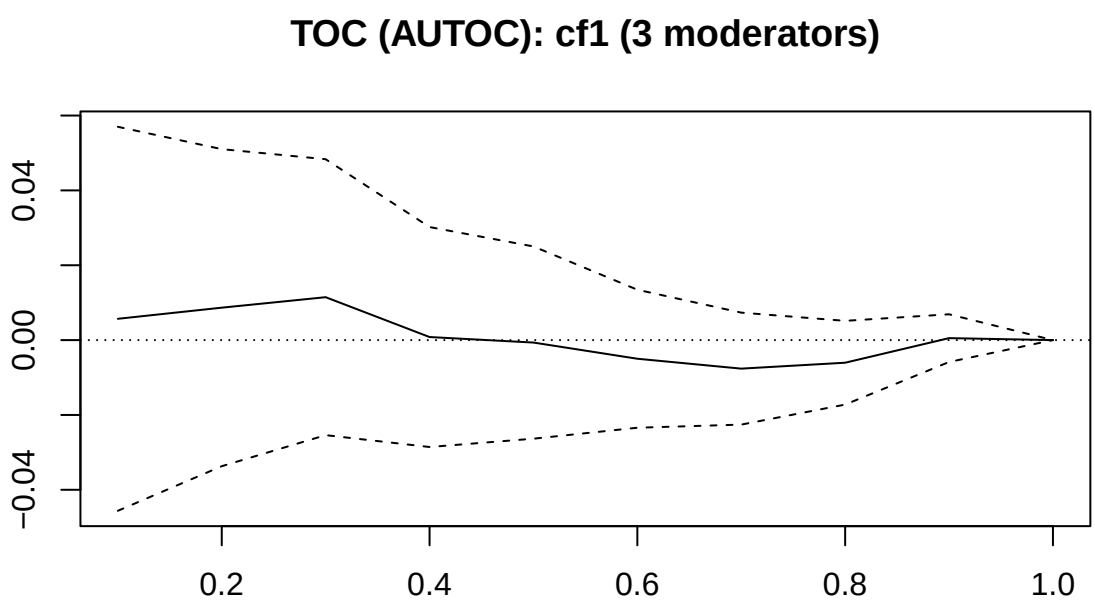
rate_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 4, caption = "RATE by forest and split seed (headline = seed 1)")
```

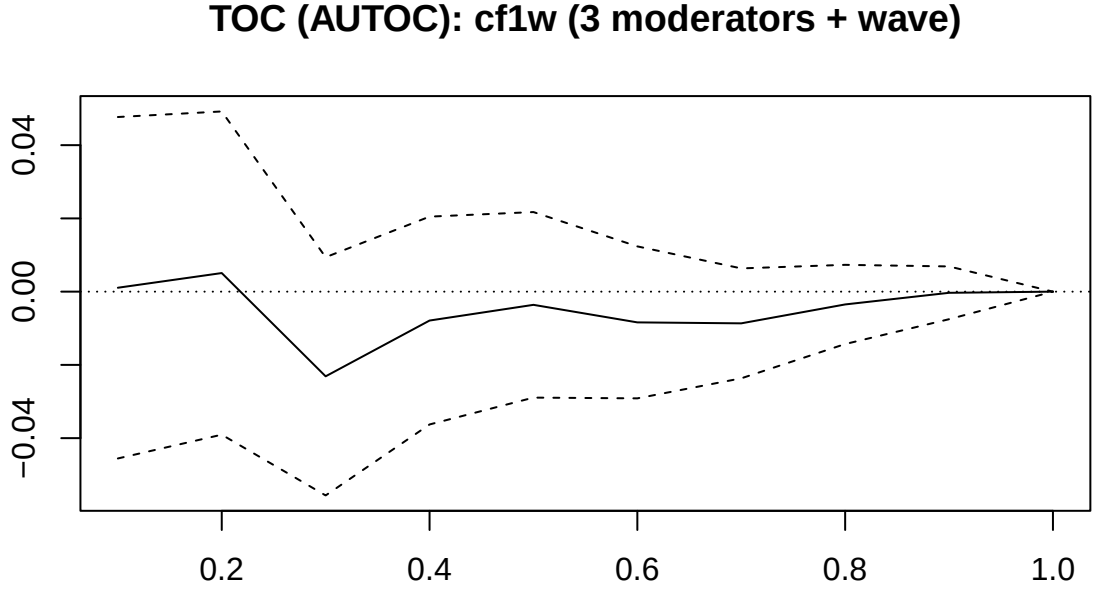
Table 8: RATE by forest and split seed (headline = seed 1)

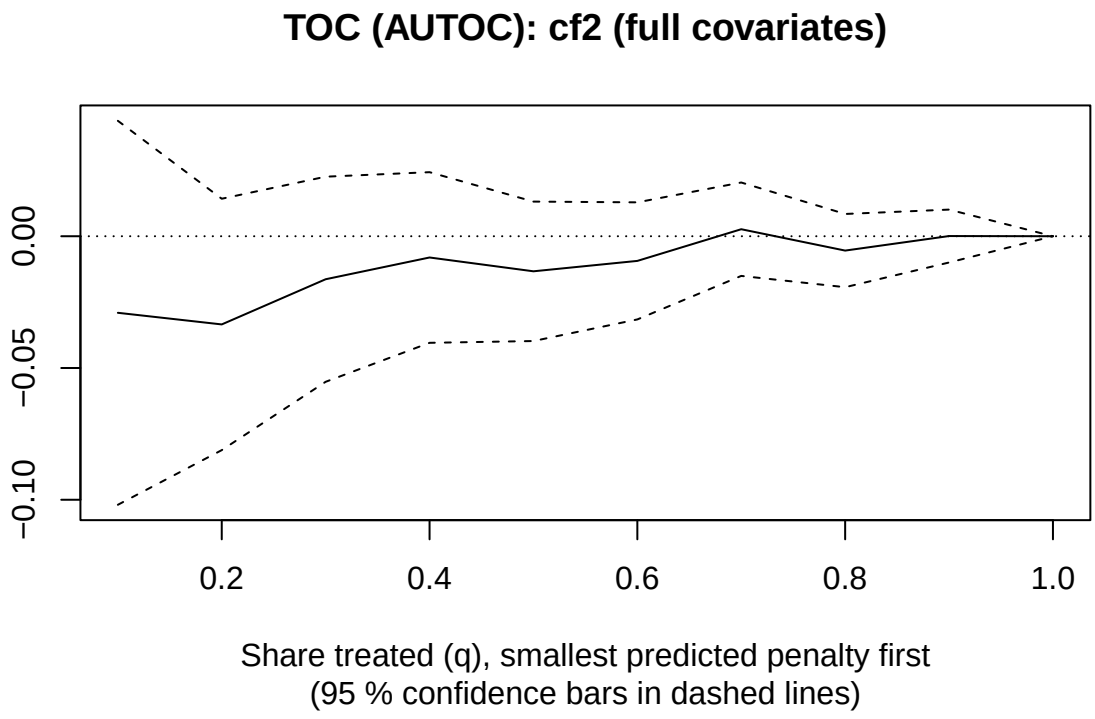
| forest | seed | headline | autoc | autoc_se | autoc_lo | autoc_hi | qini | qini_se |
|---|---|---|---|---|---|---|---|---|
| cf1 | 1 | TRUE | 0.0010 | 0.0114 | -0.0214 | 0.0233 | -0.0007 | 0.0038 |
| cf1 | 2 | FALSE | 0.0056 | 0.0108 | -0.0157 | 0.0269 | 0.0016 | 0.0033 |
| cf1 | 3 | FALSE | -0.0064 | 0.0102 | -0.0264 | 0.0136 | -0.0009 | 0.0035 |
| cf1 | 4 | FALSE | 0.0071 | 0.0103 | -0.0131 | 0.0273 | 0.0037 | 0.0033 |
| cf1 | 5 | FALSE | -0.0130 | 0.0118 | -0.0360 | 0.0101 | -0.0018 | 0.0033 |
| cf1w | 1 | TRUE | -0.0063 | 0.0102 | -0.0262 | 0.0137 | -0.0027 | 0.0036 |
| cf1w | 2 | FALSE | -0.0082 | 0.0100 | -0.0278 | 0.0114 | -0.0012 | 0.0032 |
| cf1w | 3 | FALSE | 0.0004 | 0.0107 | -0.0206 | 0.0214 | -0.0016 | 0.0035 |
| cf1w | 4 | FALSE | 0.0155 | 0.0104 | -0.0049 | 0.0360 | 0.0063 | 0.0033 |
| cf1w | 5 | FALSE | -0.0086 | 0.0110 | -0.0302 | 0.0130 | -0.0011 | 0.0034 |
| cf2 | 1 | TRUE | -0.0063 | 0.0132 | -0.0323 | 0.0197 | -0.0027 | 0.0039 |
| cf2 | 2 | FALSE | -0.0123 | 0.0111 | -0.0341 | 0.0094 | -0.0048 | 0.0031 |
| cf2 | 3 | FALSE | 0.0088 | 0.0103 | -0.0114 | 0.0290 | 0.0043 | 0.0036 |
| cf2 | 4 | FALSE | 0.0115 | 0.0117 | -0.0114 | 0.0344 | 0.0029 | 0.0033 |
| cf2 | 5 | FALSE | -0.0051 | 0.0127 | -0.0299 | 0.0198 | -0.0003 | 0.0036 |

TOC curves for the headline split. The three-moderator forests can produce only a few distinct predictions, so many resumes are tied in the ranking and the curve will look stepped; mention this in the write-up.

```
for (nm in names(rate_res)) {
  plot(rate_res[[nm]][["1"]]$autoc, main = paste("TOC (AUTOC):", nm),
       xlab = "Share treated (q), smallest predicted penalty first")
}
```


TOC (AUTOC): cf1 (3 moderators)
0.04
0.00
−0.04
0.2
0.4
0.6
0.8
1.0
Share treated (q), smallest predicted penalty first
(95 % confidence bars in dashed lines)


TOC (AUTOC): cf1w (3 moderators + wave)
0.04
0.00
−0.04
0.2
0.4
0.6
0.8
1.0
Share treated (q), smallest predicted penalty first
(95 % confidence bars in dashed lines)




### 5.3 Combined reading (rule fixed in advance)

Both checks support heterogeneity: calibration differential coefficient positive and significant (one-sided $p < 0.05$) and the headline AUTOC interval excluding 0. Neither supports it: report that we find no statistically reliable evidence of heterogeneity, and do not interpret forest subgroup patterns. They disagree: call the evidence inconclusive and treat any subgroup patterns as exploratory only. A null here may reflect low power, not constant effects.

```
verdict_tab <- cal_tab %>%
  left_join(filter(rate_tab, headline), by = "forest") %>%
  mutate(
    calibration_supports = diff_p_one_sided < 0.05,
    rate_supports        = autoc_lo > 0,
    verdict = case_when(
      calibration_supports & rate_supports   ~ "Both support heterogeneity",
      !calibration_supports & !rate_supports ~ "Neither supports: report null",
      TRUE                                   ~ "Disagree: inconclusive"
    )
  ) %>%
  select(forest, diff_est, diff_se, diff_p_one_sided,
         autoc, autoc_lo, autoc_hi, verdict)

verdict_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 3, caption = "Validation verdict by forest",
        col.names = c("Forest", "Diff. est.", "Diff. SE", "p (1-sided)",
                      "AUTOC", "AUTOC lo", "AUTOC hi", "Verdict"))
```

Table 9: Validation verdict by forest

| Forest | Diff. est. | Diff. SE | p (1-sided) | AUTOC | AUTOC lo | AUTOC hi | Verdict |
|---|---|---|---|---|---|---|---|
| cf1 | -0.053 | 0.374 | 0.556 | 0.001 | -0.021 | 0.023 | Neither supports: report null |
| cf1w | -0.185 | 0.301 | 0.731 | -0.006 | -0.026 | 0.014 | Neither supports: report null |
| cf2 | 0.022 | 0.190 | 0.454 | -0.006 | -0.032 | 0.020 | Neither supports: report null |

## 5.4 Tuning robustness

One alternative setting, fixed in advance: twice the default trees (4000 vs 2000) and a larger honesty fraction (0.7 vs the grf default 0.5). It is not a grid. The question is only whether the null verdict changes. Seed 1 is the same firm split as the headline RATE result, so the comparison is like for like.

```
alt <- list(num.trees = 4000, honesty.fraction = 0.7)

# Calibration under the alternative setting
forests_alt <- lapply(forests, function(f) {
  causal_forest(f$X, Y_all, W_all,
                clusters         = firm_id,
                num.trees        = alt$num.trees,
                honesty.fraction = alt$honesty.fraction,
                seed             = 123)
})

cal_alt <- bind_rows(lapply(names(forests_alt), function(nm) {
  tc <- test_calibration(forests_alt[[nm]])
  data.frame(
    forest           = nm,
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))

# AUTOC under the alternative setting (headline split, seed 1)
train_1 <- make_split(firm_id, 1)

rate_alt_tab <- bind_rows(lapply(names(forests), function(nm) {
  a <- run_rate(forests[[nm]]$X, Y_all, W_all, firm_id, train_1, seed = 1,
                targets          = "AUTOC",
                num.trees        = alt$num.trees,
                honesty.fraction = alt$honesty.fraction)$autoc
  data.frame(forest   = nm,
             autoc    = a$estimate,
             autoc_lo = a$estimate - 1.96 * a$std.err,
             autoc_hi = a$estimate + 1.96 * a$std.err)
}))
```

```r
# Default vs alternative
stab_tab <- cal_tab %>%
  select(forest, diff_default = diff_est, p_default = diff_p_one_sided) %>%
  left_join(select(cal_alt, forest, diff_alt = diff_est, p_alt = diff_p_one_sided),
            by = "forest") %>%
  left_join(filter(rate_tab, headline) %>%
              select(forest, autoc_default = autoc,
                     lo_default = autoc_lo, hi_default = autoc_hi),
            by = "forest") %>%
  left_join(select(rate_alt_tab, forest, autoc_alt = autoc,
                   lo_alt = autoc_lo, hi_alt = autoc_hi),
            by = "forest")

stab_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 3,
        caption = "Default vs alternative tuning (4000 trees, honesty fraction 0.7)",
        col.names = c("Forest", "Diff (def)", "p (def)", "Diff (alt)", "p (alt)",
                      "AUTOC (def)", "Lo (def)", "Hi (def)",
                      "AUTOC (alt)", "Lo (alt)", "Hi (alt)"))
```

Table 10: Default vs alternative tuning (4000 trees, honesty fraction 0.7)

| Forest | Diff (def) | p (def) | Diff (alt) | p (alt) | AUTOC (def) | Lo (def) | Hi (def) | AUTOC (alt) | Lo (alt) | Hi (alt) |
|---|---|---|---|---|---|---|---|---|---|---|
| cf1 | -0.053 | 0.556 | -0.069 | 0.571 | 0.001 | -0.021 | 0.023 | 0.002 | -0.021 | 0.024 |
| cf1w | -0.185 | 0.731 | -0.186 | 0.737 | -0.006 | -0.026 | 0.014 | -0.007 | -0.027 | 0.013 |
| cf2 | 0.022 | 0.454 | 0.035 | 0.416 | -0.006 | -0.032 | 0.020 | -0.010 | -0.036 | 0.015 |

Reading it: the verdict is stable if every forest still has a calibration p above 0.05 and an AUTOC interval that includes 0 under the alternative setting. If one forest flips, report it as sensitive to tuning and treat the evidence for that forest as inconclusive under your combined rule.